POSTGRESQL INDEXING INTERVIEW LAB
LEARNING PROCESS AND SUMMARY

Date: 2026-08-20
Topic: PostgreSQL indexing and query optimization
Mission: Prepare for a backend engineering interview by diagnosing query performance from execution-plan evidence and defending workload-driven index decisions.

1. HOW THE LEARNING PROCESS WORKED

Every exercise used the same interview loop:

1. Predict the likely bottleneck.
2. Run EXPLAIN (ANALYZE, BUFFERS), usually twice.
3. Read the plan from child nodes upward.
4. Compare estimated rows with actual rows and loops.
5. Inspect filters, buffers, sorting, and timing.
6. Propose an exact index or argue that no index is needed.
7. Only after the proposal, receive feedback and a recommendation.
8. Create the index and compare before and after plans.
9. State the tradeoff in interview language.

The database lab used PostgreSQL 16, schema db_indexes, and 500,000 synthetic wallet transfers. The initial state had only the primary-key index. Data had 10,000 wallets, skewed source-wallet activity, one year of timestamps, and status distribution approximately:

```
completed: 425,085 rows (about 85%)
pending:    50,053 rows (about 10%)
failed:     24,862 rows (about 5%)
```

SQLPad added an automatic LIMIT 10001 to result-producing queries. For full measurements in SQLPad, an aggregate such as SUM(amount_cents) was used. The outer LIMIT only limited the one aggregate result; the aggregate still had to process all matching rows.

2. EXECUTION PLAN RULES LEARNED

Read the plan tree from the deepest child upward.

Seq Scan:
Reads table pages sequentially and checks rows against a predicate.

Index Scan:
Walks an index and fetches matching heap rows, often in index order.

Bitmap Index Scan plus Bitmap Heap Scan:
Finds tuple locations in the index, builds a bitmap, then visits heap pages in physical order. This is useful when there are many matches spread across multiple pages.

Index Only Scan:
Can answer a query from index entries without visiting the heap when all required data is in the index and visibility information allows it. The count(*) workaround demonstrated Heap Fetches: 0. This was a useful extra observation, but not a main curriculum topic.

Rows:
The rows value on a node is rows emitted by that node, not rows examined.

Rows Removed by Filter shows discarded work. With loops, actual rows and removed rows are usually per-loop averages, so multiply by loops.

Cost versus time:
cost is an internal planner estimate in arbitrary units. actual time is measured milliseconds. They should not be compared as equal units.

Buffers:
shared hit means a page was already in PostgreSQL shared buffers.
shared read means PostgreSQL requested the page from the operating system.
Buffer counts often show stable work differences better than one timing.

Index Cond versus Filter:
Index Cond is used to find index entries. Filter is applied after the access step. A later condition can appear in Index Cond but still fail to narrow the contiguous range as much as a leading condition.

LIMIT:
A parent LIMIT can stop requesting rows early. EXPLAIN may still show the child estimate as if it ran to completion. Actual rows and buffers show the work that really happened before the limit stopped execution.

3. EXERCISE 1: SINGLE-COLUMN INDEX

Query:
WHERE source_wallet_id = 4242

Baseline plan:
Parallel sequential scan. The scan emitted 28 rows and removed about 499,998 rows across three worker loops. It touched about 15,156 buffers and took 27.435 ms.

Proposed index:
A single-column B-tree on source_wallet_id.

Indexed plan:
Bitmap Index Scan plus Bitmap Heap Scan. It found 28 rows, fetched 27 exact heap blocks, touched 33 buffers, and took 0.179 ms.

Measured result:
About 153 times faster and about 459 times fewer buffers.

Main lesson:
A selective equality predicate can turn a table-wide scan into a small index lookup. Estimates were already close; the main problem was access cost.

4. EXERCISE 2: COMPOSITE INDEX

Query:
WHERE source_wallet_id = 42 AND status = 'failed'

Baseline plan:
Parallel sequential scan. It returned 113 rows after examining about 500,001 rows, touched about 15,156 buffers, and took 37.760 ms.

Proposed index:
A composite B-tree on (source_wallet_id, status).

Indexed plan:
Both predicates appeared in Index Cond under a Bitmap Index Scan. The plan

returned the same 113 rows, touched 119 buffers, and took 0.402 ms.

Measured result:
About 94 times faster and about 127 times fewer buffers.

Main lesson:
A composite index can navigate directly to a frequently used combination. (source_wallet_id, status) also supports source_wallet_id alone, but does not efficiently support status alone because status is not the leading column. Separate indexes can support independent queries but may require BitmapAnd and additional heap work.

5. EXERCISE 3: COMPOSITE COLUMN ORDERING

Query:

```
WHERE source_wallet_id = 42
  AND created_at >= TIMESTAMPTZ '2025-12-01 00:00:00+00'
```

Baseline:
Parallel sequential scan. It returned 179 rows, touched about 15,156 buffers, and took 45.863 ms.

Equality-first ordering:
(source_wallet_id, created_at)
179 rows, 185 buffers, 0.490 ms.

Range-first ordering:
(created_at, source_wallet_id)
179 rows, 346 buffers, 2.377 ms. The index scan used about 168 index buffers instead of about 7 for equality-first ordering.

Main lesson:
For a multicolumn B-tree, leading equality conditions usually narrow the contiguous index range first. The first range condition then limits the remaining scan. A later condition can still appear in Index Cond while doing less to reduce the range traversed.

Interview wording:
Put leading equality conditions before the first range condition when that matches the workload. Do not reduce the rule to "highest cardinality first."

6. EXERCISE 4: ORDER BY PLUS LIMIT

Query:

```
WHERE status = 'completed'
ORDER BY created_at DESC, id DESC
LIMIT 50
```

Baseline plan:
Parallel sequential scan, Gather Merge, and top-N heapsort. It considered about 425,085 completed rows, touched about 15,244 buffers, and took 34.053 ms.

Proposed index:
(status, created_at DESC, id DESC)

Indexed plan:
Limit over an Index Scan. No Sort and no Gather Merge. It returned 50 rows, touched 53 buffers, and took 0.211 ms.

Measured result:
  About 161 times faster and about 288 times fewer buffers.

Main lesson:
  LIMIT reduces returned data, but cannot avoid filtering and sorting unless an
  index can provide both the predicate access and requested order. Include id
  as a deterministic tie-breaker when the query orders by id.

7. EXERCISE 5: LOW-SELECTIVITY COLUMN

Query:
  WHERE status = 'failed'

Full baseline:
  About 24,862 rows matched and about 475,138 rows were rejected. The table
  scan touched 15,156 buffers.

Status index:
  The count(*) version used an Index Only Scan with Heap Fetches: 0, because
  the query only needed a count and the index contained status. This was a
  special projection effect.

Heap-touching version:
  SUM(amount_cents) forced heap access. PostgreSQL used Bitmap Index Scan plus
  Bitmap Heap Scan, with 24 index buffers and about 12,254 exact heap blocks.
  It took 17.129 ms.

Main lesson:
  Low cardinality does not automatically make an index useless. Query
  selectivity, row width, heap-page distribution, cache state, and workload
  frequency decide whether it pays off. A 5% subset can justify an index even
  when an 85% subset should use a sequential scan.

8. EXERCISE 6: OFFSET VERSUS KEYSET PAGINATION

Query shape:
  WHERE source_wallet_id = 42
  ORDER BY created_at DESC, id DESC

Unindexed early page:
  Parallel sequential scan plus top-N sort. 50 rows took 27.126 ms and
  touched 15,244 buffers.

Unindexed deep page:
  OFFSET 1500 LIMIT 50. The query still scanned the table and sort input, then
  passed 1,550 rows through Gather Merge before returning 50. It used quicksort,
  about 215 KiB per worker, took 31.661 ms, and touched 15,244 buffers.

Ordering index:
  (source_wallet_id, created_at DESC, id DESC)
  The deep offset plan became an Index Scan with no Sort or Gather Merge. It
  walked 1,550 index entries, touched 1,564 buffers, and took 3.008 ms.

Keyset cursor:
  (created_at, id) < (cursor_created_at, cursor_id)
  The same index started at the cursor, returned 50 rows, touched 54 buffers,
  and took 0.450 ms.

Measured result:
  Keyset used about 29 times fewer buffers and was about 6.7 times faster than

the indexed deep-offset query.

Main lesson:
Offset pagination walks and discards earlier entries. Keyset pagination uses an ordered cursor, usually including a unique tie-breaker, and work stays close to page size instead of growing with page depth.

9. EXERCISE 7: CORRECT SEQUENTIAL SCAN

Query:
WHERE status = 'completed'

Full aggregate evidence:
About 425,085 rows matched and about 74,915 rows were rejected. The plan used a Parallel Seq Scan, touched 15,156 buffers, and took 92.567 ms for the aggregate.

The status index existed, but PostgreSQL correctly ignored it.

Main lesson:
An index is an option, not a command. When a query returns most of a table, using the index can require many heap-page visits. A sequential scan can read those pages more efficiently in physical order.

Important correction:
The completed query returned about 85%, not all rows. The failed query was selective enough to benefit from the same status index.

10. EXERCISE 8: WRITE-PERFORMANCE TRADEOFFS

The benchmark inserted 25,000 deterministic rows inside a transaction and rolled the transaction back. Index maintenance still occurred during the measured COPY. Index bytes include the primary key.

Baseline, primary key only:
elapsed=67.229 ms
rows/second=371,864
index-bytes=11.3 MiB

Useful read indexes:
elapsed=247.263 ms
rows/second=101,107
index-bytes=54.3 MiB

Excessive set:
elapsed=309.471 ms
rows/second=80,783
index-bytes=75.6 MiB

Compared with baseline, the useful set made writes about 3.7 times slower and added 43 MiB of index storage. The excessive set made writes about 4.6 times slower and added about 64 MiB of storage.

Final workload-backed set:

(status, created_at DESC, id DESC)
(source_wallet_id, created_at DESC, id DESC)

Removed for this workload:

```
  (source_wallet_id, status)
  (status, amount_cents)

Nuance:
  (source_wallet_id, status) could still be justified if that exact query is
  frequent and its read savings exceed its maintenance cost, even when the
  source/time index exists.

Interview rule:
  Start with measured workload frequency, predicate selectivity, ordering, and
  limit behavior. Add an index when its read or write-path benefit justifies
  its storage and maintenance cost. Validate with EXPLAIN ANALYZE and a write
  benchmark instead of relying on a column rule alone.


11. CORRECTIONS AND RETRIEVAL NOTES

  - Rows emitted are not rows scanned. Use Rows Removed by Filter and loops.
  - shared hit and shared read are buffer observations, not exact disk counts.
  - An index does not change cardinality estimates; ANALYZE statistics do.
  - Bitmap scanning is a planner strategy, not a heap requirement.
  - Index Cond can include a later condition without narrowing the leading
    index range as much as an equality-first ordering.
  - LIMIT can make actual child rows much lower than the child estimate.
  - Keyset pagination uses one index seek plus the next page entries, not one
    row total.
  - A status index can help failed queries and still be ignored for completed.
  - SQLPad may add LIMIT 10001; aggregate queries avoid truncating the scan.


12. HOW TO RESUME

Lab directory:
  /Users/tommy/projects/research/.worktrees/db-indexes/db-indexes

Open lesson 1:
  open lessons/0001-reading-explain.html

Start PostgreSQL from the repository root:
  docker compose up -d postgres

Reset the lab to its initial index state:
  go run . reset-indexes

Run the full exercise workbook in SQLPad or psql:
  exercises.sql

Use the answer gate for every new query:
  plan nodes; estimated versus actual rows; loops; filters; buffers; sort;
  planning and execution time; planner rationale; proposed index or no index.

Ask the teaching agent to continue from the next unanswered exercise. Do not
ask for a recommendation until the investigation and proposal are written.


13. PRIMARY SOURCES

  https://www.postgresql.org/docs/16/using-explain.html
  https://www.postgresql.org/docs/16/indexes-intro.html
  https://www.postgresql.org/docs/16/indexes-multicolumn.html
  https://www.postgresql.org/docs/16/indexes-ordering.html
  https://www.postgresql.org/docs/16/indexes-bitmap-scans.html
```

End of learning record.